Control Statements: Part 2

Chapter 6 of Visual C# How to Program, 6/e

OBJECTIVES

In this chapter you'll:

- Learn the essentials of counter-controlled iteration.
- Use the `for` and `do...while` iteration statements to execute statements in a program repeatedly.
- Understand multiple selection using the `switch` selection statement.
- Use the `break` and `continue` program-control statements to alter the flow of control.
- Use the logical operators to form compound conditions in control statements.
- Understand short-circuit evaluation of the conditional `&&` and `||` operators.
- Understand the representational errors associated with using floating-point data types to hold monetary values.
- See a summary of structured programming and C#'s control statements

6.2 Essentials of Counter-Controlled Iteration

- Counter-controlled iteration requires
  - a **control variable** (or loop counter)
  - the **initial value** of the control variable
  - the **increment** (or **decrement**) by which the control variable is modified each **iteration of the loop**
  - the **loop-continuation condition**
- Fig. 6.1 uses a loop to display the numbers from 1 through 10.

```
1  // Fig. 6.1: WhileCounter.cs
2  // Counter-controlled iteration with the while iteration statement.
3  using System;
4
5  class WhileCounter
6  {
7     static void Main()
8     {
9        int counter = 1; // declare and initialize control variable
10
11       while (counter <= 10) // loop-continuation condition
12       {
13          Console.Write($"{counter}  ");
14          ++counter; // increment control variable
15       }
16
17       Console.WriteLine();
18    }
19 }
```

```
1  2  3  4  5  6  7  8  9  10
```

Fig. 6.1 | Counter-controlled iteration with the `while` iteration statement.

Error-Prevention Tip 6.1

Floating-point values are approximate, so controlling counting loops with floating-point variables of types `float` *or* `double` *can result in imprecise counter values and inaccurate tests for termination. Use integer values to control counting loops.*

6.3 `for` Iteration Statement

- The **`for` iteration statement** specifies the elements of counter-controlled-iteration in a single line of code.
- Figure 6.2 reimplements the app using the `for` statement.

```
1  // Fig. 6.2: ForCounter.cs
2  // Counter-controlled iteration with the for iteration statement.
3  using System;
4
5  class ForCounter
6  {
7     static void Main()
8     {
9        // for statement header includes initialization,
10       // loop-continuation condition and increment
11       for (int counter = 1; counter <= 10; ++counter)
12       {
13          Console.Write($"{counter}  ");
14       }
15
16       Console.WriteLine();
17    }
18 }
```

```
1  2  3  4  5  6  7  8  9  10
```

Fig. 6.2 | Counter-controlled iteration with the `for` iteration statement.

Common Programming Error 6.1

Using an incorrect relational operator or an incorrect final value of a loop counter in the loop-continuation condition of an iteration statement can cause an off-by-one error.

Error-Prevention Tip 6.2

Using the final value and operator `<=` *in a loop's condition helps avoid off-by-one errors. For a loop that outputs 1 to 10, the loop-continuation condition should be* `counter <= 10` *rather than* `counter < 10` *(which causes an off-by-one error) or* `counter < 11` *(which is correct). Many programmers prefer zero-based counting in which, to count 10 times, you'd initialize* `counter` *to zero and use the loop-continuation test* `counter < 10`.

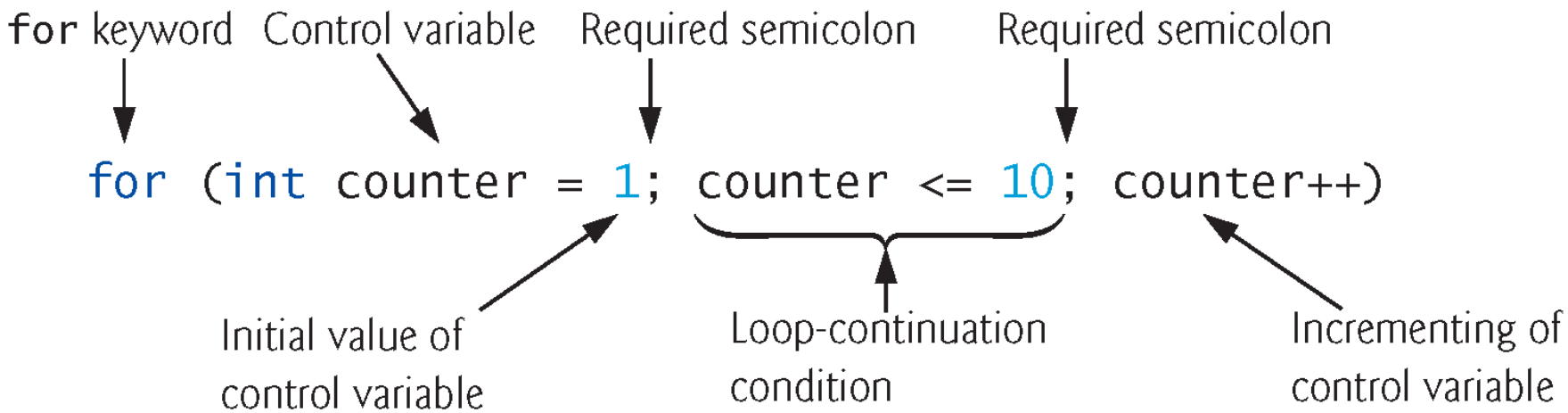


Fig. 6.3 | `for` statement header components.

6.3 `for` Iteration Statement (Cont.)

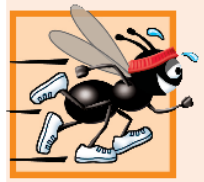
- If the *initialization* expression declares the control variable, the control variable will not exist outside the `for` statement.
- This restriction is known as the variable's **scope**.
- Similarly, a local variable can be used only in the method that declares the variable and only from the point of declaration.

Common Programming Error 6.2

When a `for` *statement's control variable is declared in the initialization section of a* `for`*'s header, using the control variable after the* `for`*'s body is a compilation error.*

6.3 `for` Iteration Statement (Cont.)

- All three expressions in a `for` header are optional.
  - Omitting the *loopContinuationCondition* creates an infinite loop.
  - You can omit the *initialization* expression if the control variable is initialized before the loop.
  - You can omit the *increment* expression if the app calculates the increment with statements in the loop's body or if no increment is needed.



Performance Tip 6.1

There's a slight performance advantage to using the prefix increment operator, but if you choose the postfix increment operator because it seems more natural (as in a `for` *header), optimizing compilers are likely to generate code that uses the more efficient form anyway.*

Error-Prevention Tip 6.3

Infinite loops occur when the loop-continuation condition in an iteration statement never becomes false. *To prevent this situation in a counter-controlled loop, ensure that the control variable is incremented (or decremented) during each iteration of the loop. In a sentinel-controlled loop, ensure that the sentinel value is eventually entered.*

6.3 `for` Iteration Statement (Cont.)

- The initialization, loop-continuation condition and increment portions of a `for` statement can contain arithmetic expressions.
- For example, assume that `x = 2` and `y = 10`; if `x` and `y` are not modified in the body of the loop, then the statement

```
for (int j = x; j <= 4 * x * y; j += y / x)
```

is equivalent to the statement

```
for (int j = 2; j <= 80; j += 5)
```

- If the loop-continuation condition is initially `false`, the app does not execute the `for` statement's body.

Error-Prevention Tip 6.4

Although the value of the control variable can be changed in the body of a `for` *loop, avoid doing so, because this practice can lead to subtle errors. If a program must modify the control variable's value in the loop's body, use* `while` *rather than* `for`.

6.3 `for` Iteration Statement (Cont.)

- Fig. 6.4 shows the activity diagram of the for statement in Fig. 6.2.

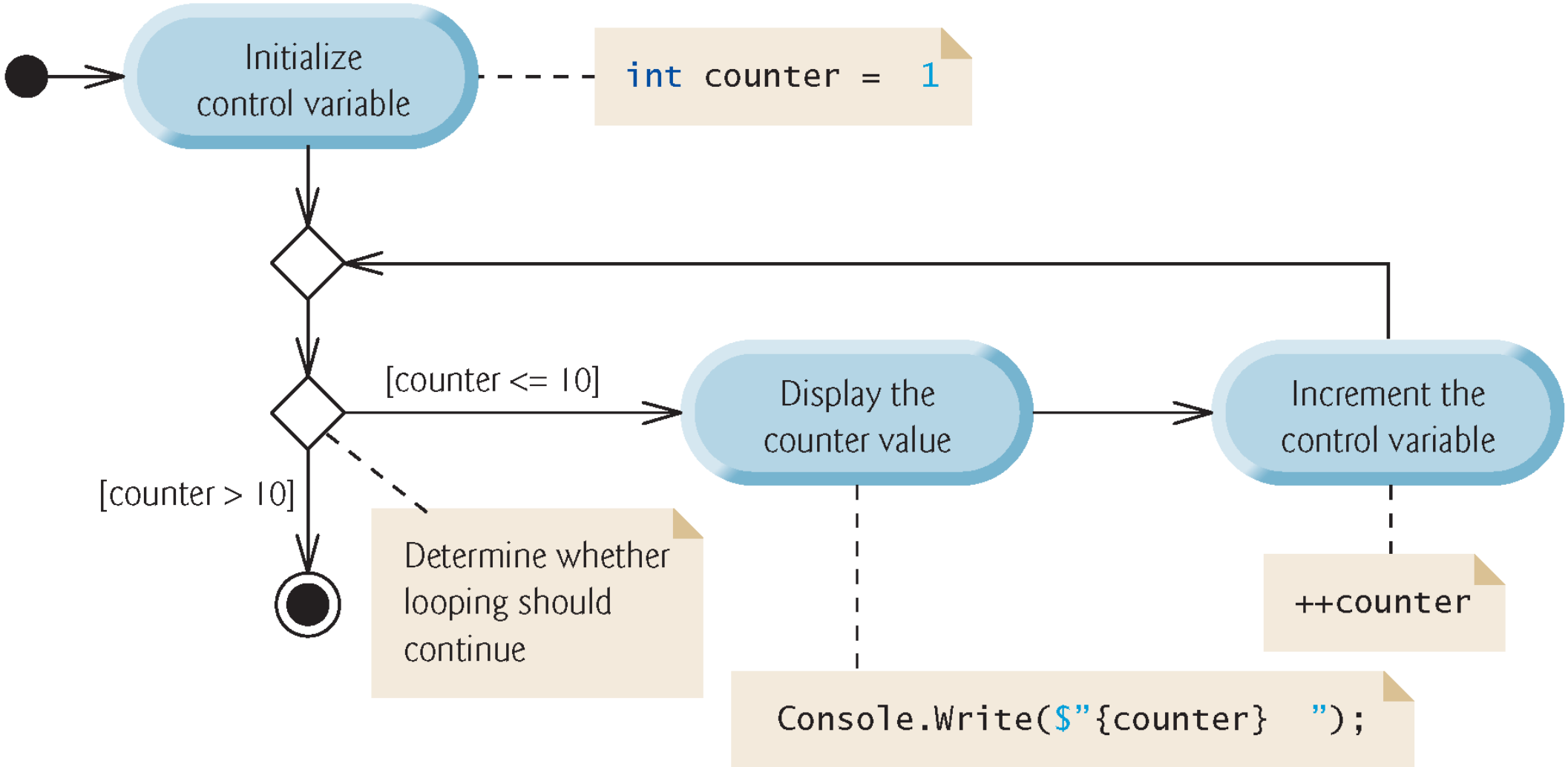


Fig. 6.4 | UML activity diagram for the `for` statement in Fig. 6.2.

6.4 Examples Using the `for` Statement

Common Programming Error 6.3

Using an incorrect relational operator in the loop-continuation condition of a loop that counts downward (e.g., using `i <= 1` *instead of* `i >= 1` *in a loop counting down to 1) is usually a logic error.*

Common Programming Error 6.4

Do not use equality operators (`!=` or `==`) in a loop-continuation condition if the loop's control variable increments or decrements by more than 1. For example, consider the `for` *statement header* `for (int counter = 1; counter != 10; counter += 2)`. *The loop-continuation test* `counter != 10` *never becomes false (resulting in an* infinite loop*) because* `counter` *increments by 2 after each iteration (and never becomes 10).*

6.5 App: Summing Even Integers

- The app in Fig. 6.5 uses a `for` statement to sum the even integers from 2 to 20 and store the result in an `int` variable called `total`.

```
1  // Fig. 6.5: Sum.cs
2  // Summing integers with the for statement.
3  using System;
4
5  class Sum
6  {
7     static void Main()
8     {
9        int total = 0; // initialize total
10
11       // total even integers from 2 through 20
12       for (int number = 2; number <= 20; number += 2)
13       {
14          total += number;
15       }
16
17       Console.WriteLine($"Sum is {total}"); // display results
18    }
19 }
```

```
Sum is 110
```

Fig. 6.5 | Summing integers with the `for` statement.

6.5 App: Summing Even Integers

- The *initialization* and *increment* expressions can be *comma-separated lists* that enable you to use multiple initialization expressions or multiple increment expressions.
- For example, although this is discouraged, you could merge the for's body statement (line 14) into the increment portion of the for header by using a comma as follows:
  - `total += number, number += 2`

6.6 App: Compound-Interest Calculations

- A person invests $1,000 in a savings account yielding 5% interest, compounded yearly. Calculate and display the amount of money in the account at the end of each year for 10 years.

$$a = p\,(1 + r)^n$$

p is the original amount invested (i.e., the principal)
r is the annual interest rate (use 0.05 for 5%)
n is the number of years
a is the amount on deposit at the end of the *n*th year.

Fig. 6.6 uses a loop that performs the calculation for each of the 10 years the money remains on deposit.

```
1  // Fig. 6.6: Interest.cs
2  // Compound-interest calculations with for.
3  using System;
4
5  class Interest
6  {
7     static void Main()
8     {
9        decimal principal = 1000; // initial amount before interest
10       double rate = 0.05; // interest rate
11
12       // display headers
13       Console.WriteLine("Year   Amount on deposit");
14
```

Fig. 6.6 | Compound-interest calculations with `for`. (Part 1 of 3.)

```
15          // calculate amount on deposit for each of ten years
16          for (int year = 1; year <= 10; ++year)
17          {
18             // calculate new amount for specified year
19             decimal amount = principal *
20                ((decimal) Math.Pow(1.0 + rate, year));
21
22             // display the year and the amount
23             Console.WriteLine($"{year,4}{amount,20:C}");
24          }
25       }
26    }
```

Fig. 6.6 | Compound-interest calculations with `for`. (Part 2 of 3.)

```
Year   Amount on deposit
   1          $1,050.00
   2          $1,102.50
   3          $1,157.63
   4          $1,215.51
   5          $1,276.28
   6          $1,340.10
   7          $1,407.10
   8          $1,477.46
   9          $1,551.33
  10          $1,628.89
```

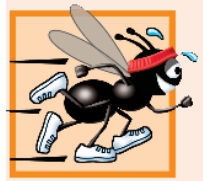
Fig. 6.6 | Compound-interest calculations with `for`. (Part 3 of 3.)

6.6.1 Performing the Interest Calculations with `Math` Method `pow`

- Many classes provide static methods that cannot be called on objects—they must be called using a class name.
- You can call a `static` method by specifying the class name followed by the member access (`.`) operator and the method name:

  ClassName.*methodName*(*arguments*)

- `Math.Pow(x, y)` calculates the value of *x* raised to the *y*th power.



Performance Tip 6.2

In loops, avoid calculations for which the result never changes—such calculations should typically be placed before the loop. Optimizing compilers will typically do this for you.

6.6.2 Formatting with Field Widths and Alignment

- `Console.WriteLine($"{year,4}{amount,20:C}");`
  - displays the year and the amount on deposit at the end of that year.
- The interpolation expression
  - `{year,4}`
- formats the year.
  - The integer 4 after the comma indicates that the value output should be displayed with a field width of 4.
  - If the value to be output is fewer than four character positions wide (one or two characters in this example), the value is right-aligned in the field by default.
  - If the value to be output were more than four character positions wide, the field width would be extended to the right to accommodate the entire value.

6.6.2 Formatting with Field Widths and Alignment

- `Console.WriteLine($"{year,4}{amount,20:C}");`
  - displays the year and the amount on deposit at the end of that year.
- The interpolation expression
  - `{year,4}`
- formats the year.
  - The integer 4 after the comma indicates that the value output should be displayed with a field width of 4.
  - If the value to be output is fewer than four character positions wide (one or two characters in this example), the value is right-aligned in the field by default.
  - If the value to be output were more than four character positions wide, the field width would be extended to the right to accommodate the entire value.

Common Programming Error 6.5

Using floating-point numbers in a manner that assumes they're represented exactly (e.g., using them in comparisons for equality) can lead to incorrect results. Floating-point numbers are represented only approximately.

6.6.3 Caution: Do Not Use `float` or `double` for Monetary Amounts

- Section 4.9 introduced the simple type `decimal` for precise monetary representation and calculations.
- For certain values types `float` and `double` suffer from what we call representational error.
  - Floating-point numbers often arise as a result of calculations—when we divide 10 by 3, the result is 3.3333333…, with the sequence of 3s repeating infinitely.
  - The computer allocates only a fixed amount of space to hold such a value, so clearly the stored floating-point value can be only an approximation.

Error-Prevention Tip 6.5

Do not use variables of type `double` *(or* `float`*) to perform precise monetary calculations—use type* `decimal` *instead. The imprecision of floating-point numbers can cause errors that will result in incorrect monetary values.*

6.7 `do...while` Iteration Statement

- The **`do...while` iteration statement** (Fig. 6.7) is similar to the `while` statement.

```
1  // Fig. 6.7: DoWhileTest.cs
2  // do...while iteration statement.
3  using System;
4
5  class DoWhileTest
6  {
7     static void Main()
8     {
9        int counter = 1; // initialize counter
10
11       do
12       {
13          Console.Write($"{counter}  ");
14          ++counter;
15       } while (counter <= 10); // required semicolon
16
17       Console.WriteLine();
18    }
19 }
```

```
1  2  3  4  5  6  7  8  9  10
```

Fig. 6.7 | do...while iteration statement.

6.7 `do...while` Iteration Statement

- Figure 6.8 contains the UML activity diagram for the `do...while` statement.

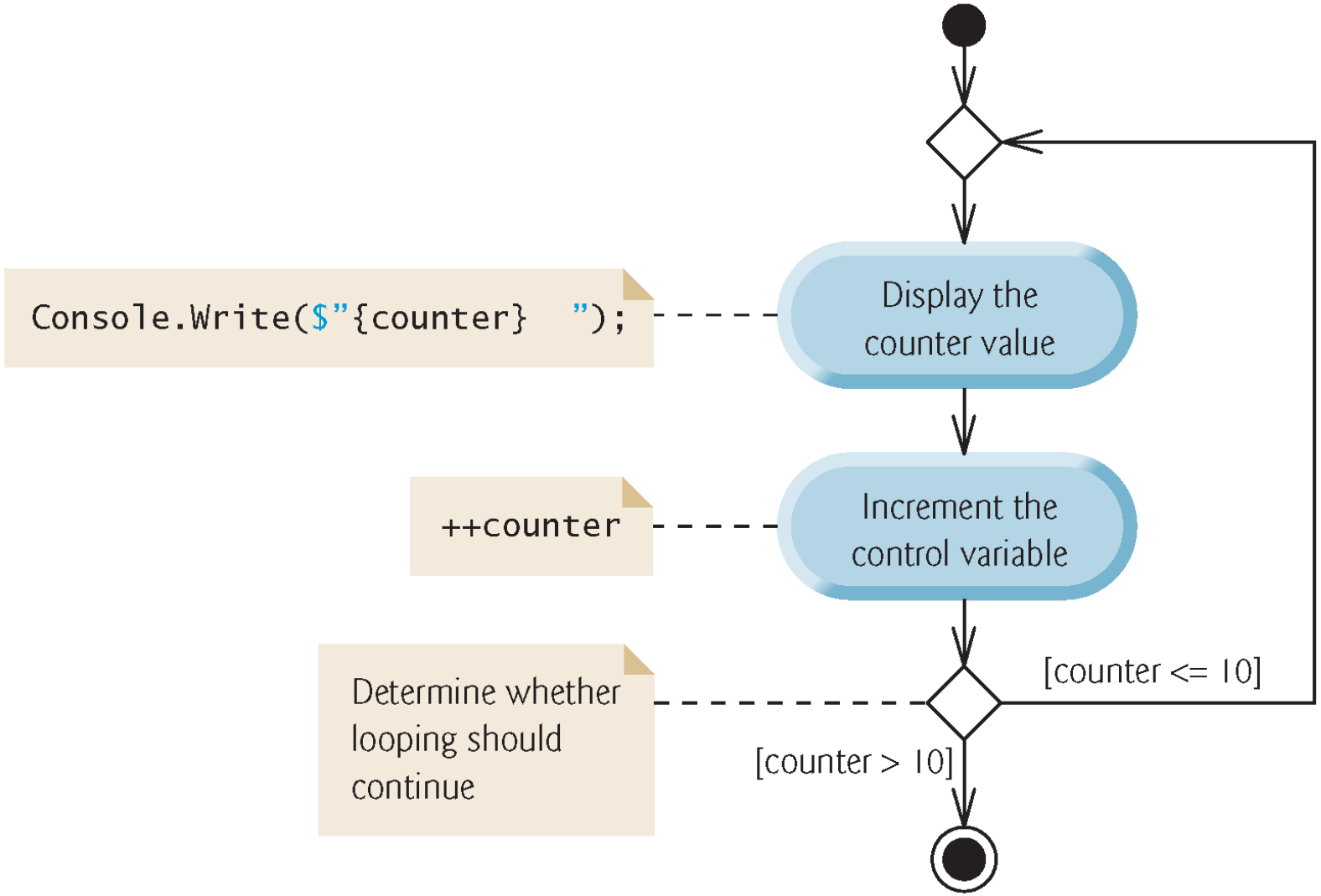


Fig. 6.8 | `do...while` iteration statement UML activity diagram.

6.8 `switch` Multiple-Selection Statement

- The **`switch` multiple-selection** statement performs different actions based on the value of an expression.
- Each action is associated with the value of a **constant integral expression** or a **constant string expression** that the expression may assume.

6.8.1 Using a `switch` Statement to Count A, B, C, D and F Grades

- Figure 6.9 calculates the class average of a set of numeric grades entered by the user, and uses a `switch` statement to determine whether each grade is the equivalent of an A, B, C, D or F and to increment the appropriate grade counter.
- The program also displays a summary of the number of students who received each grade.

```
1  // Fig. 6.9: LetterGrades.cs
2  // Using a switch statement to count letter grades.
3  using System;
4
5  class LetterGrades
6  {
7     static void Main()
8     {
9        int total = 0; // sum of grades
10       int gradeCounter = 0; // number of grades entered
11       int aCount = 0; // count of A grades
12       int bCount = 0; // count of B grades
13       int cCount = 0; // count of C grades
14       int dCount = 0; // count of D grades
15       int fCount = 0; // count of F grades
16
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 1 of 6.)

```
17          Console.WriteLine("Enter the integer grades in the range 0-100.");
18          Console.WriteLine(
19             "Type <Ctrl> z and press Enter to terminate input:");
20
21          string input = Console.ReadLine(); // read user input
22
23          // loop until user enters the end-of-file indicator (<Ctrl> z)
24          while (input != null)
25          {
26             int grade = int.Parse(input); // read grade off user input
27             total += grade; // add grade to total
28             ++gradeCounter; // increment number of grades
29
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 2 of 6.)

```
30            // determine which grade was entered
31            switch (grade / 10)
32            {
33               case 9: // grade was in the 90s
34               case 10: // grade was 100
35                  ++aCount; // increment aCount
36                  break; // necessary to exit switch
37               case 8: // grade was between 80 and 89
38                  ++bCount; // increment bCount
39                  break; // exit switch
40               case 7: // grade was between 70 and 79
41                  ++cCount; // increment cCount
42                  break; // exit switch
43               case 6: // grade was between 60 and 69
44                  ++dCount; // increment dCount
45                  break; // exit switch
46               default: // grade was less than 60
47                  ++fCount; // increment fCount
48                  break; // exit switch
49            }
50
51            input = Console.ReadLine(); // read user input
52         }
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 3 of 6.)

```
53
54          Console.WriteLine("\nGrade Report:");
55
56          // if user entered at least one grade...
57          if (gradeCounter != 0)
58          {
59             // calculate average of all grades entered
60             double average = (double) total / gradeCounter;
61
62             // output summary of results
63             Console.WriteLine(
64                $"Total of the {gradeCounter} grades entered is {total}");
65             Console.WriteLine($"Class average is {average:F}");
66             Console.WriteLine("Number of students who received each grade:");
67             Console.WriteLine($"A: {aCount}"); // display number of A grades
68             Console.WriteLine($"B: {bCount}"); // display number of B grades
69             Console.WriteLine($"C: {cCount}"); // display number of C grades
70             Console.WriteLine($"D: {dCount}"); // display number of D grades
71             Console.WriteLine($"F: {fCount}"); // display number of F grades
72          }
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 4 of 6.)

```
73          else // no grades were entered, so output appropriate message
74          {
75             Console.WriteLine("No grades were entered");
76          }
77       }
78    }
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 5 of 6.)

```
Enter the integer grades in the range 0-100.
Type <Ctrl> z and press Enter to terminate input:
99
92
45
57
63
71
76
85
90
100
^Z

Grade Report:
Total of the 10 grades entered is 778
Class average is 77.80
Number of students who received each grade:
A: 4
B: 1
C: 2
D: 1
F: 2
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 6 of 6.)

6.8.1 Using a `switch` Statement to Count A, B, C, D and F Grades

- The expression following keyword `switch` is called the **`switch` expression**.
- The app attempts to match the value of the `switch` expression with one of the `case` labels.
- You are required to include a statement that terminates the `case`, such as a `break`, a `return` or a `throw`.
- The **`break` statement** causes program control to proceed with the first statement after the `switch`.
- If no match occurs, the statements after the `default` label execute.

Good Programming Practice 6.1

Although each `case` *and the* `default` *label in a* `switch` *can occur in any order, place the* `default` *label last for clarity.*

6.8.2 `switch` Statement UML Activity Diagram

- Figure 6.11 shows the UML activity diagram for the general `switch` statement.

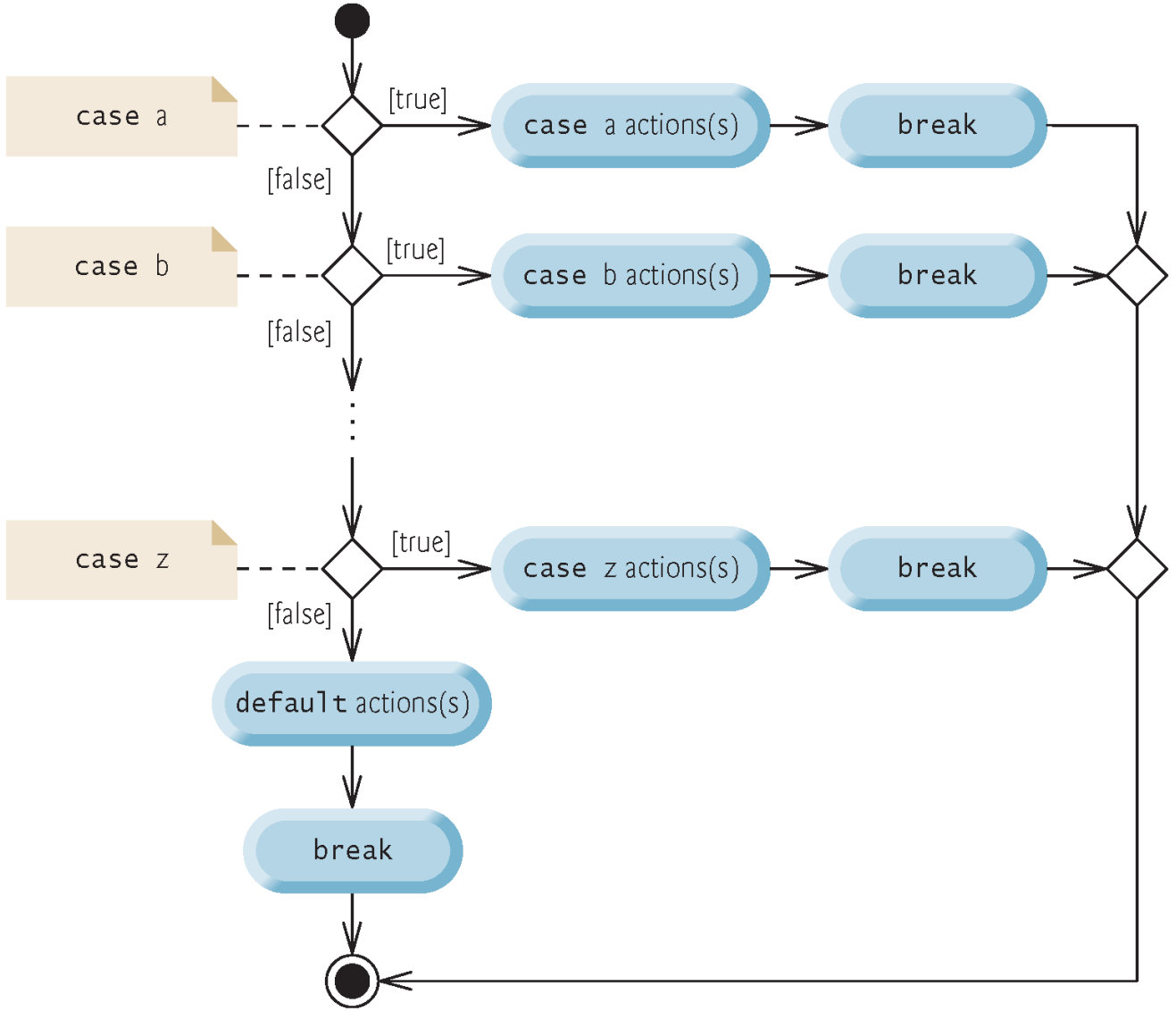


Fig. 6.10 | `switch` multiple-selection statement UML activity diagram with `break` statements.

6.8.3 Notes on the Expression in Each `case` of a `switch`

- The expression after each `case` can be only a constant integral expression or a constant string expression.
- You can also use `null` and **character constants** which represent the integer values of characters.
- The expression also can be a **constant** that contains a value which does not change for the entire app.

6.9 Class `AutoPolicy` Case Study: `strings` in `switch` Statements

- `strings` can be used in `switch` expressions, and `string` literals can be used in `case` labels.
- *You've been hired by an auto insurance company that serves these northeast states—Connecticut, Maine, Massachusetts, New Hampshire, New Jersey, New York, Pennsylvania, Rhode Island and Vermont. The company would like you to create a program that produces a report indicating for each of their auto insurance policies whether the policy is held in a state with "no-fault" auto insurance—Massachusetts, New Jersey, New York and Pennsylvania.*

```
1  // Fig. 6.11: AutoPolicy.cs
2  // Class that represents an auto insurance policy.
3  class AutoPolicy
4  {
5     public int AccountNumber { get; set; } // policy account number
6     public string MakeAndModel { get; set; } // car that policy applies to
7     public string State { get; set; } // two-letter state abbreviation
8
9     // constructor
10    public AutoPolicy(int accountNumber, string makeAndModel, string state)
11    {
12       AccountNumber = accountNumber;
13       MakeAndModel = makeAndModel;
14       State = state;
15    }
16
```

Fig. 6.11 | Class that represents an auto insurance policy. (Part 1 of 2.)

```
17      // returns whether the state has no-fault insurance
18      public bool IsNoFaultState
19      {
20         get
21         {
22            bool noFaultState;
23
24            // determine whether state has no-fault auto insurance
25            switch (State) // get AutoPolicy object's state abbreviation
26            {
27               case "MA": case "NJ": case "NY": case "PA":
28                  noFaultState = true;
29                  break;
30               default:
31                  noFaultState = false;
32                  break;
33            }
34
35            return noFaultState;
36         }
37      }
38   }
```

Fig. 6.11 | Class that represents an auto insurance policy. (Part 2 of 2.)

```
1  // Fig. 6.12: AutoPolicyTest.cs
2  // Demonstrating strings in switch.
3  using System;
4
5  class AutoPolicyTest
6  {
7     static void Main()
8     {
9        // create two AutoPolicy objects
10       AutoPolicy policy1 = new AutoPolicy(11111111, "Toyota Camry", "NJ");
11       AutoPolicy policy2 = new AutoPolicy(22222222, "Ford Fusion", "ME");
12
13       // display whether each policy is in a no-fault state
14       PolicyInNoFaultState(policy1);
15       PolicyInNoFaultState(policy2);
16    }
17
```

Fig. 6.12 | Demonstrating `strings` in `switch`. (Part 1 of 2.)

```
18      // method that displays whether an AutoPolicy
19      // is in a state with no-fault auto insurance
20      public static void PolicyInNoFaultState(AutoPolicy policy)
21      {
22         Console.WriteLine("The auto policy:");
23         Console.Write($"Account #: {policy.AccountNumber}; ");
24         Console.WriteLine($"Car: {policy.MakeAndModel};");
25         Console.Write($"State {policy.State}; ");
26         Console.Write($"({policy.IsNoFaultState ? "is": "is not"})");
27         Console.WriteLine(" a no-fault state\n");
28      }
29   }
```

```
The auto policy:
Account #: 11111111; Car: Toyota Camry;
State NJ is a no-fault state

The auto policy:
Account #: 22222222; Car: Ford Fusion;
State ME is not a no-fault state
```

Fig. 6.12 | Demonstrating `strings` in `switch`. (Part 2 of 2.)

6.10.1 `break` Statement

- The `break` statement causes immediate exit from a statement.
- Figure 6.13 demonstrates a `break` statement exiting a `for`.

```
1  // Fig. 6.13: BreakTest.cs
2  // break statement exiting a for statement.
3  using System;
4
5  class BreakTest
6  {
7     static void Main()
8     {
9        int count; // control variable also used after loop terminates
10
11       for (count = 1; count <= 10; ++count) // loop 10 times
12       {
13          if (count == 5) // if count is 5,
14          {
15             break; // terminate loop
16          }
17
18          Console.Write($"{count} ");
19       }
20
21       Console.WriteLine($"\nBroke out of loop at count = {count}");
22    }
23 }
```

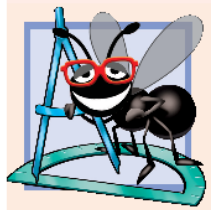
Fig. 6.13 | break statement exiting a for statement. (Part 1 of 2.)

```
1 2 3 4
Broke out of loop at count = 5
```

Fig. 6.13 | `break` statement exiting a `for` statement. (Part 2 of 2.)

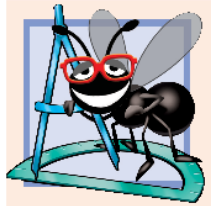
6.10.2 `break` Statement

- The **`continue` statement** skips the remaining statements in the loop body and tests whether to proceed with the next iteration of the loop.
- In a `for` statement, the increment expression executes, then the app evaluates the loop-continuation test.
- Figure 6.14 uses `continue` in a `for` statement.



Software Engineering Observation 6.1

Some programmers feel that `break` *and* `continue` *statements violate structured programming. Since the same effects are achievable with structured-programming techniques, these programmers prefer not to use* `break` *or* `continue` *statements.*



Software Engineering Observation 6.2

There's a tension between achieving quality software engineering and achieving the best-performing software. Often, one of these goals is achieved at the expense of the other. For all but the most performance-intensive situations, apply the following rule: First, make your code simple and correct; then make it fast, but only if necessary.

```
1  // Fig. 6.14: ContinueTest.cs
2  // continue statement skipping an iteration of a for statement.
3  using System;
4
5  class ContinueTest
6  {
7     static void Main()
8     {
9        for (int count = 1; count <= 10; ++count) // loop 10 times
10       {
11          if (count == 5) // if count is 5,
12          {
13             continue; // skip remaining code in loop
14          }
15
16          Console.Write($"{count} ");
17       }
18
19       Console.WriteLine("\nUsed continue to skip displaying 5");
20    }
21 }
```

Fig. 6.14 | `continue` statement skipping an iteration of a `for` statement. (Part 1 of 2.)

```
1 2 3 4 6 7 8 9 10
Used continue to skip displaying 5
```

Fig. 6.14 | `continue` statement skipping an iteration of a `for` statement. (Part 2 of 2.)

6.11 Logical Operators

- Simple conditions are expressed in terms of the relational operators >, <, >= and <=, and the equality operators == and !=.
  - Each expression tests only one condition.
- C# provides **logical operators** to enable you to form more complex conditions by combining simple conditions.
- The logical operators are && (conditional AND), || (conditional OR), & (boolean logical AND), | (boolean logical inclusive OR), ^ (boolean logical exclusive OR) and ! (logical negation).

6.11.1 Conditional AND (&&) Operator

- The **&&** (**conditional AND**) operator works as follows:

```
if (gender == 'F' && age >= 65)
{
   ++seniorFemales;
}
```

- The combined condition is true if and only if *both* simple conditions are true.
- The combined condition may be more readable with redundant parentheses:

```
(gender == 'F') && (age >= 65)
```

6.11.1 Conditional AND (&&) Operator

- The table in Fig. 6.15 shows all four possible combinations of `false` and `true` values for *expression1* `and` *expression2*.
- Such tables are called **truth tables**.

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 6.15 | && (conditional AND) operator truth table.

6.11.2 Conditional OR (||) Operator

- The **||** (**conditional OR**) operator, as in the following app segment:

```
if ((semesterAverage >= 90) || (finalExam >= 90))
{
   Console.WriteLine ("Student grade is A");
}
```

- The complex condition evaluates to `true` if either or both of the simple conditions are true.
- Figure 6.16 is a truth table for operator conditional OR (||).

expression1	expression2	expression1 || expression2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 6.16 | || (conditional OR) operator truth table.

6.11.3 Short-Circuit Evaluation of Complex Conditions

- The parts of an expression containing && or || operators are evaluated only until it's known whether the condition is true or false.
  - `(gender == 'F') & (age >= 65)`
- If `gender` is not equal to `'F'` (i.e., it is certain that the entire expression is `false`) evaluation stops.
- This feature is called **short-circuit evaluation**.

Common Programming Error 6.6

In expressions using operator `&&`*, a condition—known as the dependent condition—may require another condition to be true for the evaluation of the dependent condition to be meaningful. In this case, the dependent condition should be placed* after *the other one, or an error might occur. For example, in the expression* `(i != 0) && (10 / i == 2)`*, the second condition must appear after the first condition, or a divide-by-zero error might occur.*

6.11.4 Boolean Logical AND (&) and Boolean Logical OR (|) Operators

- The **boolean logical AND** (**&**) and **boolean logical inclusive OR** (**|**) operators do not perform short-circuit evaluation.
- This is useful if the right operand has a required **side effect**. For example:

```
(birthday == true) | (++age >= 65)
```

- This ensures that the condition `++age >= 65` will be evaluated.

Error-Prevention Tip 6.6

For clarity, avoid expressions with side effects in conditions. The side effects may appear clever, but they can make it harder to understand code and can lead to subtle logic errors.

6.11.5 Boolean Logical Exclusive OR (^)

- A complex condition containing the **boolean logical exclusive OR** (`^`) operator (also called the **logical XOR operator**) is `true` *if and only if one of its operands is* `true` *and the other is* `false`.
- Figure 6.17 is a truth table for the boolean logical exclusive OR operator (^).

expression1	expression2	expression1 ^ expression2
false	false	false
false	true	true
true	false	true
true	true	false

Fig. 6.17 | ^ (boolean logical exclusive OR) operator truth table.

6.11.6 Logical Negation (!) Operator

- The ! (**logical negation** or **not**) operator enables you to "reverse" the meaning of a condition.
  - ```
    if (! (grade == sentinelValue))
    {
       Console.WriteLine($"The next grade is {grade}");
    }
    ```
- In most cases, you can avoid using logical negation by expressing the condition differently:
  - ```
    if (grade != sentinelValue)
    {
       Console.WriteLine($"The next grade is {grade}");
    }
    ```
- Figure 6.18 is a truth table for the logical negation operator.

expression	!expression
`false`	`true`
`true`	`false`

Fig. 6.18 | `!` (logical negation) operator truth table.

6.11.7 Logical Operators Example

- Figure 6.19 demonstrates the logical operators and boolean logical operators.

```
1  // Fig. 6.19: LogicalOperators.cs
2  // Logical operators.
3  using System;
4
5  class LogicalOperators
6  {
7     static void Main()
8     {
9        // create truth table for && (conditional AND) operator
10       Console.WriteLine("Conditional AND (&&)");
11       Console.WriteLine($"false && false: {false && false}");
12       Console.WriteLine($"false && true: {false && true}");
13       Console.WriteLine($"true && false: {true && false}");
14       Console.WriteLine($"true && true: {true && true}\n");
15
16       // create truth table for || (conditional OR) operator
17       Console.WriteLine("Conditional OR (||)");
18       Console.WriteLine($"false || false: {false || false}");
19       Console.WriteLine($"false || true: {false || true}");
20       Console.WriteLine($"true || false: {true || false}");
21       Console.WriteLine($"true || true: {true || true}\n");
22
```

Fig. 6.19 | Logical operators. (Part 1 of 5.)

```
23          // create truth table for & (boolean logical AND) operator
24          Console.WriteLine("Boolean logical AND (&)");
25          Console.WriteLine($"false & false: {false & false}");
26          Console.WriteLine($"false & true: {false & true}");
27          Console.WriteLine($"true & false: {true & false}");
28          Console.WriteLine($"true & true: {true & true}\n");
29
30          // create truth table for | (boolean logical inclusive OR) operator
31          Console.WriteLine("Boolean logical inclusive OR (|)");
32          Console.WriteLine($"false | false: {false | false}");
33          Console.WriteLine($"false | true: {false | true}");
34          Console.WriteLine($"true | false: {true | false}");
35          Console.WriteLine($"true | true: {true | true}\n");
36
```

Fig. 6.19 | Logical operators. (Part 2 of 5.)

```
37          // create truth table for ^ (boolean logical exclusive OR) operator
38          Console.WriteLine("Boolean logical exclusive OR (^)");
39          Console.WriteLine($"false ^ false: {false ^ false}");
40          Console.WriteLine($"false ^ true: {false ^ true}");
41          Console.WriteLine($"true ^ false: {true ^ false}");
42          Console.WriteLine($"true ^ true: {true ^ true}\n");
43
44          // create truth table for ! (logical negation) operator
45          Console.WriteLine("Logical negation (!)");
46          Console.WriteLine($"!false: {!false}");
47          Console.WriteLine($"!true: {!true}");
48      }
49  }
```

Fig. 6.19 | Logical operators. (Part 3 of 5.)

```
Conditional AND (&&)
false && false: False
false && true: False
true && false: False
true && true: True

Conditional OR (||)
false || false: False
false || true: True
true || false: True
true || true: True

Boolean logical AND (&)
false & false: False
false & true: False
true & false: False
true & true: True
```

Fig. 6.19 | Logical operators. (Part 4 of 5.)

```
Boolean logical inclusive OR (|)
false | false: False
false | true: True
true | false: True
true | true: True

Boolean logical exclusive OR (^)
false ^ false: False
false ^ true: True
true ^ false: True
true ^ true: False

Logical negation (!)
!false: True
!true: False
```

Fig. 6.19 | Logical operators. (Part 5 of 5.)

6.11.7 Logical Operators Example

- Figure 6.20 shows the C# operators from highest precedence to lowest.

Operators	Associativity	Type
`.` `new` `++`*(postfix)* `--`*(postfix)*	left to right	highest precedence
`++` `--` `+` `-` `!` (*type*)	right to left	unary prefix
`*` `/` `%`	left to right	multiplicative
`+` `-`	left to right	additive
`<` `<=` `>` `>=`	left to right	relational
`==` `!=`	left to right	equality
`&`	left to right	boolean logical AND
`^`	left to right	boolean logical exclusive OR
`|`	left to right	boolean logical inclusive OR
`&&`	left to right	conditional AND
`||`	left to right	conditional OR
`?:`	right to left	conditional
`=` `+=` `-=` `*=` `/=` `%=`	right to left	assignment

Fig. 6.20 | Precedence/associativity of the operators discussed so far.

6.12 Structured-Programming Summary

- Structured programming produces apps that are easier to understand, test, debug, and modify.
- The final state of one control statement is connected to the initial state of the next (Fig. 6.21). We call this "control-statement stacking."

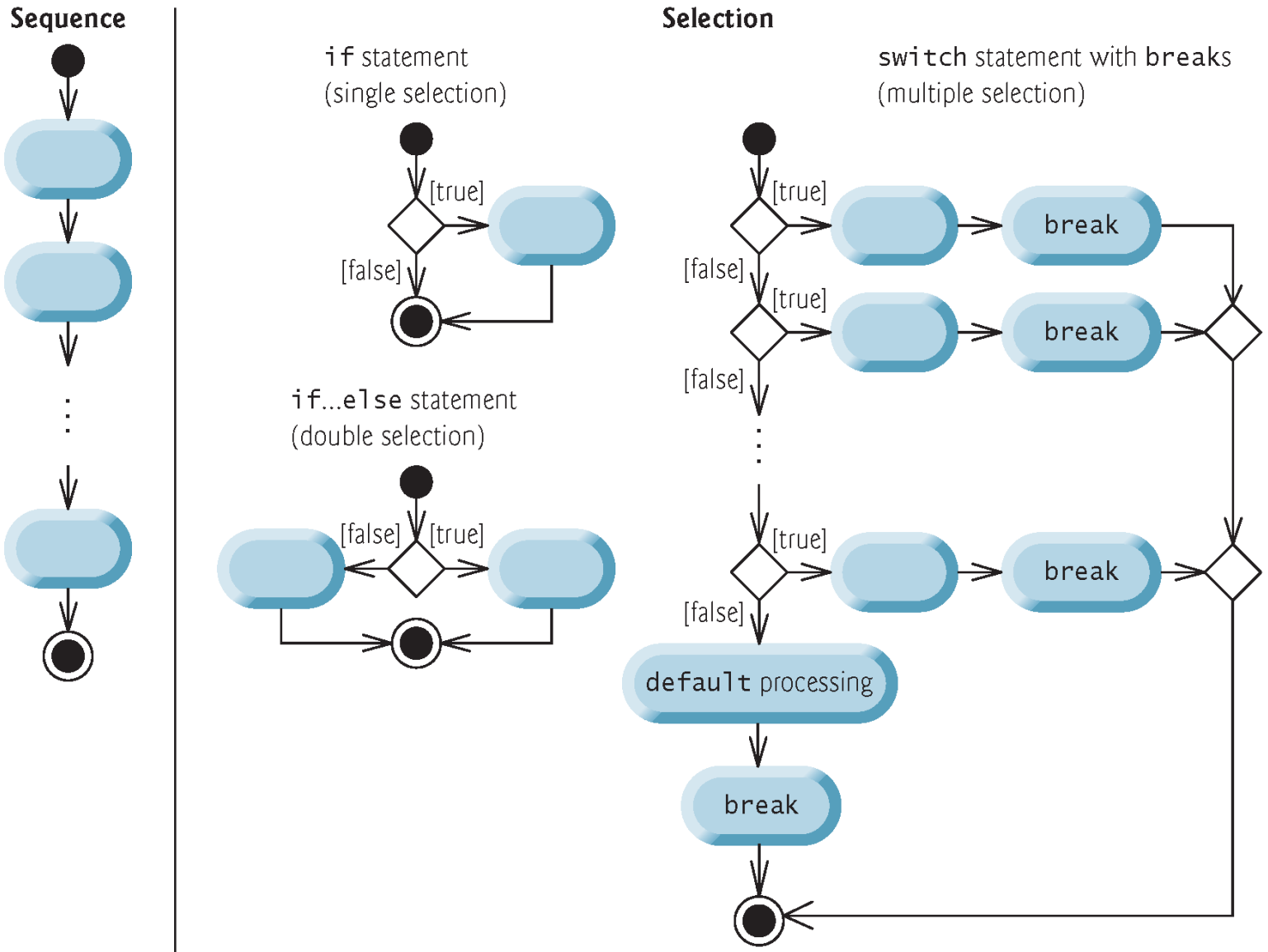


Fig. 6.21 | C#'s sequence, selection and iteration statements. (Part 1 of 2.)

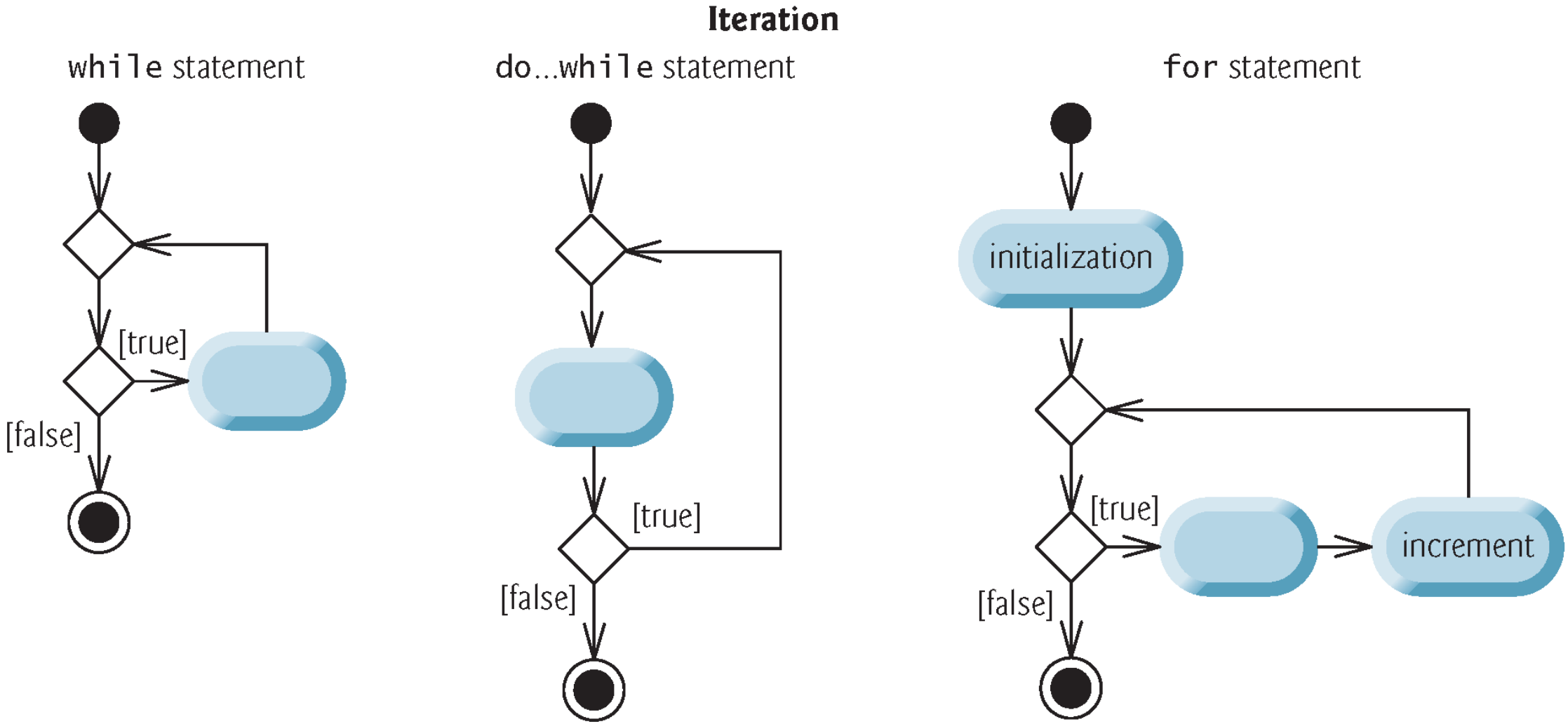


Fig. 6.21 | C#'s sequence, selection and iteration statements. (Part 2 of 2.)

Rules for forming structured apps

1. Begin with the simplest activity diagram (Fig. 6.23).
2. Any action state can be replaced by two action states in sequence.
3. Any action state can be replaced by any control statement (sequence of action states, `if`, `if...else`, `switch`, `while`, `do...while`, `for` or `foreach`, which we'll see in Chapter 8).
4. Rules 2 and 3 can be applied as often as necessary in any order.

Fig. 6.22 | Rules for forming structured apps.

6.12 Structured-Programming Summary

- We begin with the simplest activity diagram (Fig. 6.23) consisting of only an initial state, an action state, a final state and transition arrows.

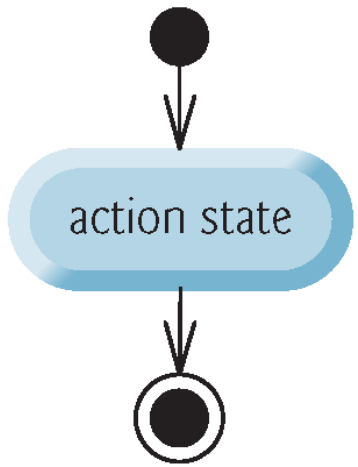


Fig. 6.23 | Simplest activity diagram.

6.12 Structured-Programming Summary

- Rule 2 generates a stack of control statements, so let us call rule 2 the **stacking rule**.
- Repeatedly applying rule 2 results in an activity diagram containing many action states in sequence (Fig. 6.24).

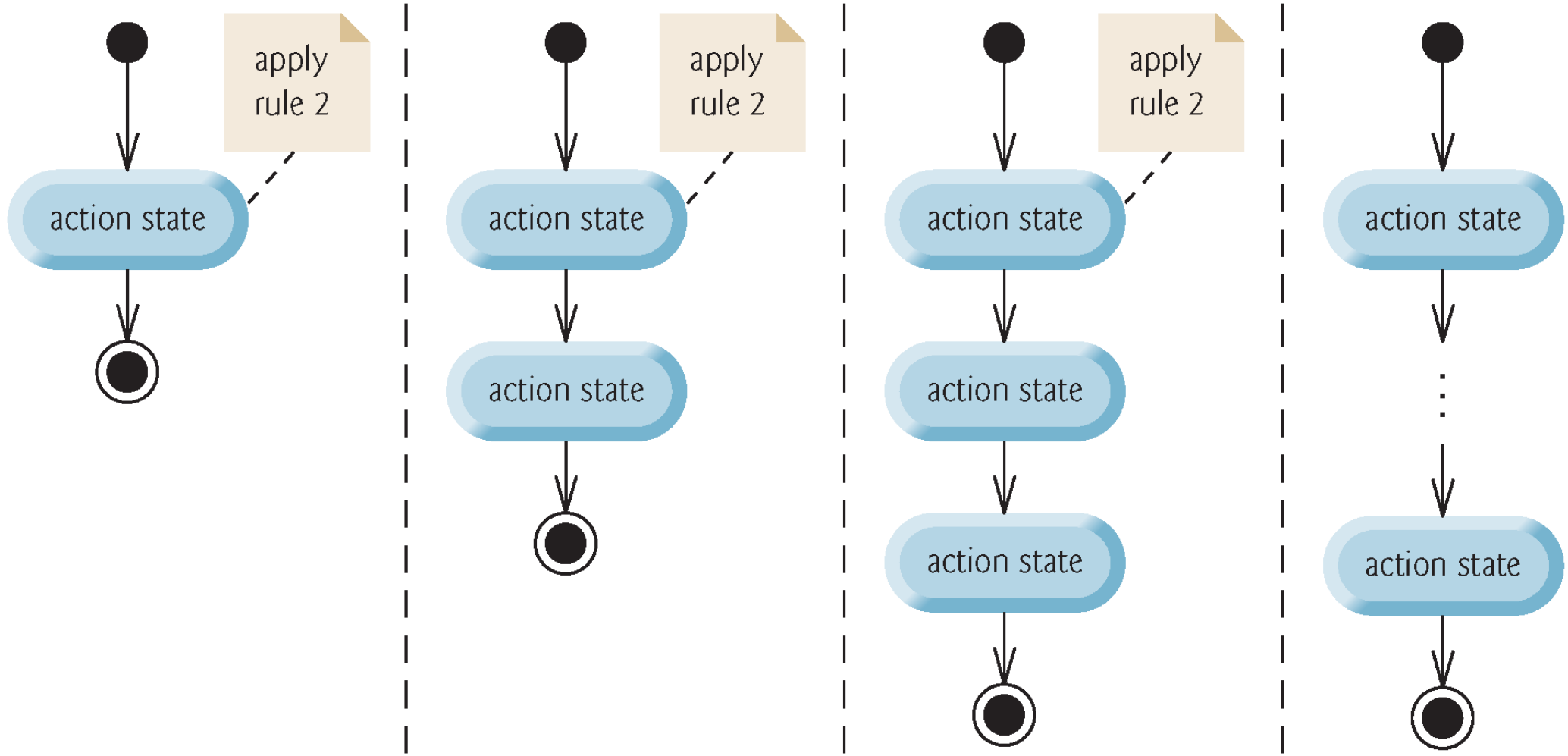


Fig. 6.24 | Repeatedly applying the stacking rule (Rule 2) of Fig. 6.22 to the simplest activity diagram.

6.12 Structured-Programming Summary

- Rule 3 is called the **nesting rule**.
- Repeatedly applying rule 3 to the simplest activity diagram results in an activity diagram with neatly **nested control statements**.

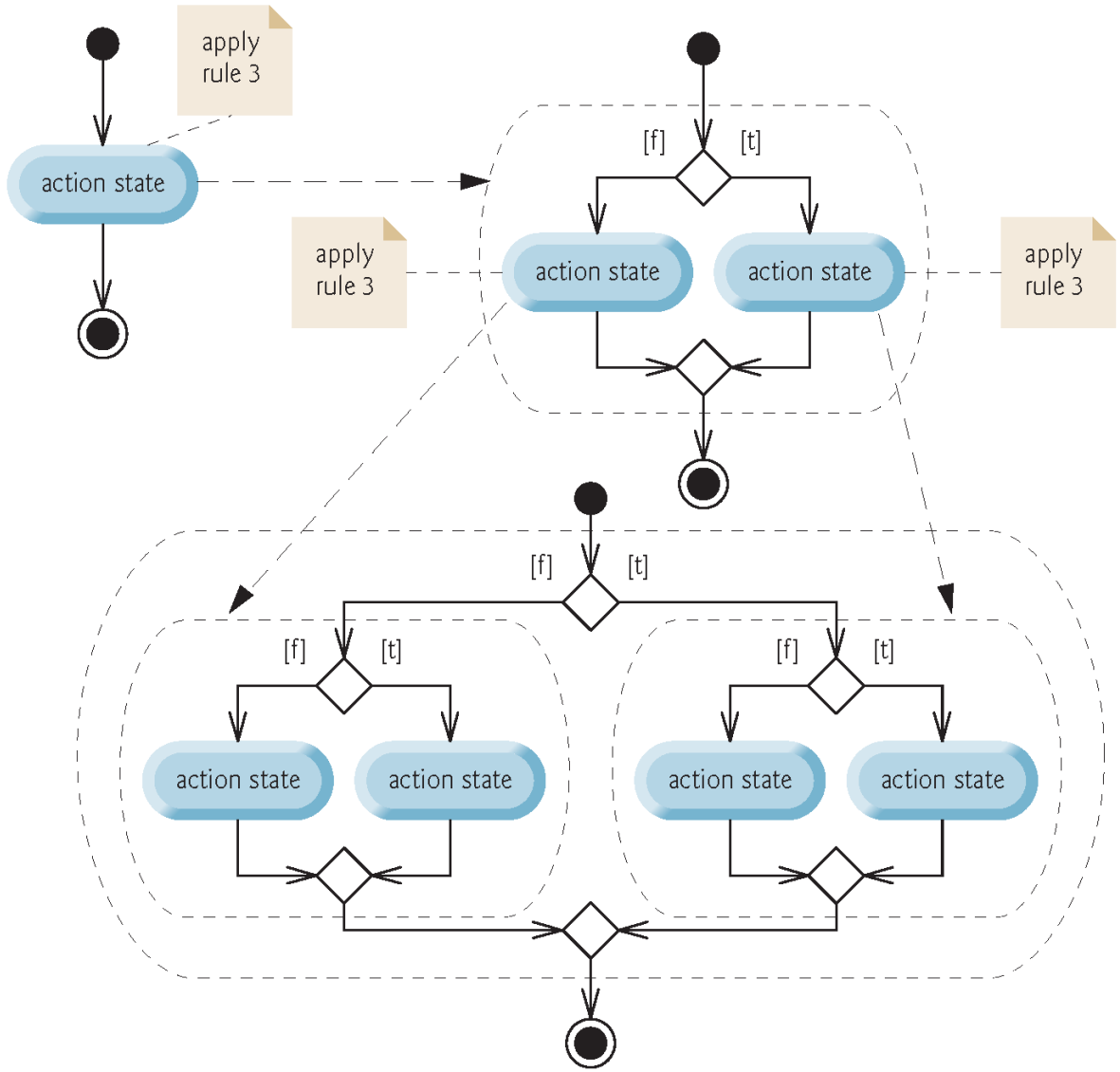


Fig. 6.25 | Repeatedly applying Rule 3 of Fig. 6.22 to the simplest activity diagram.

6.12 Structured-Programming Summary

- Rule 4 generates larger, more involved and more deeply nested statements.
- If the rules are followed, an “unstructured” activity diagram (like the one in Fig. 6.26) cannot be created.

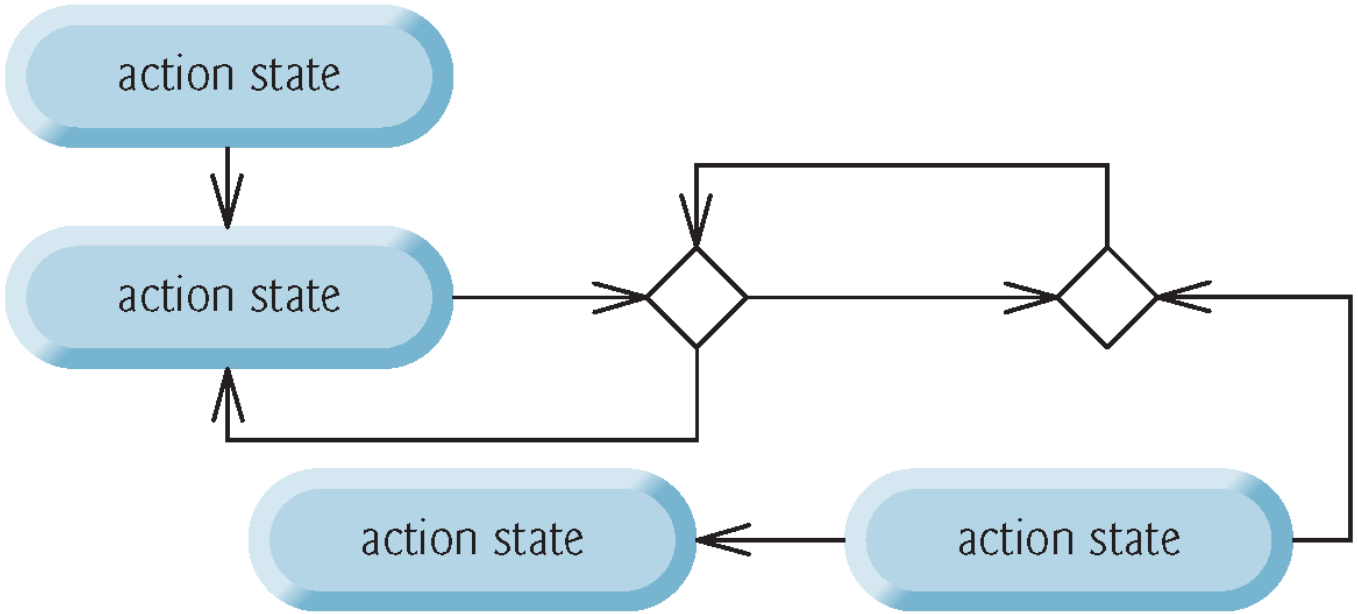


Fig. 6.26 | "Unstructured" activity diagram.